Analysing FLIC Data: Options from the Pletcher Laboratory

For users of the FLIC (Fly Liquid-food Interaction Counter) system · October 2026

The FLIC system records each fly's contact with food: the Drosophila Feeding Monitors (DFMs) measure the signal from every feeding well, and the MCU, running PyMCU, saves those signals as data files. Turning the files into feeding measures, figures and statistics takes analysis software. The Pletcher Laboratory offers two packages for this. Both are available at no cost from the laboratory's GitHub page:

- **FLIC R Code**, the laboratory's original analysis code, written in R.
- **pyflic**, its successor, a Python application with a graphical interface.

We recommend pyflic for all new work, and we encourage laboratories that use the R code to move to it.

The two options at a glance

	FLIC R Code	**pyflic**
What it is	A set of R functions and example scripts	A desktop application, plus a Python library for those who want one
How you use it	You write R scripts in RStudio	You point and click in the Analysis Hub; scripting is optional
Skills needed	A working knowledge of R	None beyond following a step-by-step install guide
Quality control	Individual functions you call yourself	Built-in checks, a QC Viewer and a QC section in every report
Figures and reports	Plots you build in code	Publication-ready figures and PDF reports, made for you
Help	Three worked-example documents	Help on every screen (press F1), plus user guides
Where to get it	github.com/PletcherLab/FLIC_R_Code	github.com/PletcherLab/pyflic
Development	Remains available; the laboratory's new development is in pyflic	Actively developed: new analyses and support for new hardware arrive here

Option 1: FLIC R Code

The R code is the analysis the laboratory has used for many years. It counts the fly's interactions with food (licks and feeding events) in single-well and two-well choice experiments, and computes summaries and preference indices from them.

Where to get it: https://github.com/PletcherLab/FLIC_R_Code. You will need R, RStudio and the R packages *ggplot2*, *reshape2* and *gridExtra*. The repository's Readme explains the setup, and its three example documents (*GettingStarted*, *GroupedAnalysis* and *ChoiceChamberAnalysis*) walk through typical analyses with example data. The code is released under the MIT licence.

What to expect: the R code has no graphical interface. Every step, including loading data, checking quality, describing the experimental design and drawing figures, is a line of R that you write and maintain. It suits users who are comfortable in R, but it becomes unwieldy as experiments grow larger or more numerous.

Option 2: pyflic (recommended)

pyflic is the laboratory's Python successor to the R code. It uses the same method for finding feeding in the signal, with the same parameters, and it is designed to reproduce the R code's results. Around that core it adds a graphical interface, built-in quality control, pooled analysis of several experiments, and automatic reports.

Where to get it: https://github.com/PletcherLab/pyflic. The *Getting Started* guide (https://github.com/PletcherLab/pyflic/blob/main/docs/GETTING_STARTED.md) assumes no experience with Python or the command line and takes about 15 minutes. A free tool called *uv* installs Python and everything pyflic needs, kept apart from anything else on the computer. pyflic runs on Windows, macOS and Linux, and needs no network connection, database or server.

Why move to pyflic

- **No programming required.** An experiment is described once in a form, and analyses run from checkboxes and a single button.
- **Problems are caught before they reach your results.** pyflic checks every recording for gaps, drifting signals, interference between the two wells of a choice chamber and chambers that should be excluded, and shows you the evidence.
- **It scales.** Several related experiments can be pooled into one *Project* with combined statistics, and a whole folder of Projects can run unattended.
- **Results are reproducible.** Every setting is saved in a readable text file beside the data, and each report lists the parameters it used.
- **Moving over is straightforward.** The parameters are those of the R code (`Feeding.Threshold` becomes `feeding_threshold`), and the pyflic README includes a table matching R functions to their pyflic equivalents.
- **It is where development happens.** New analyses, and support for new versions of the FLIC hardware and software, are being added to pyflic.

Features

- **Analysis Hub:** one window to load data, run quality control, analyse and plot, with an editor for setting up each experiment and its treatments.
- **Feeding and tasting detection:** licks, feeding events, tasting, bout durations, intervals between bouts, and preference indices for two-well choice experiments.
- **Quality control:** a data-integrity report, a QC Viewer that shows the raw signal beside the feeding pyflic detected, checks for interference between wells, and exclusion rules that record why each chamber was left out.

- **Assay types:** ready-made analyses for Hedonic feeding and Progressive Ratio (including the breaking point), a general-purpose type, and an editor in which a laboratory can define its own assay types without writing code.
- **Optogenetics:** a light QC that compares when each LED was on with when the fly was actually feeding, and flags light that ran with no feeding behind it (usually a sign of a sensor problem rather than fly behaviour).
- **Statistics:** ANOVA and linear mixed models, bootstrap confidence intervals, parameter sensitivity analyses and the fine structure of feeding bouts. Pooled Projects add a mixed model with DFM nested within experiment.
- **Figures and reports:** journal-ready vector figures from a Plot Editor, and PDF reports for each experiment and each Project covering quality control, results, statistics and the settings used.
- **Automation:** saved analysis scripts that rerun identically, and batch runs across many Projects.
- **Built-in help:** press F1 or a ? button anywhere for the relevant topic; every detection parameter links to an explanation of what it does.
- **Optional extras:** a Python library for use in Jupyter notebooks, and an optional AI-written summary of a Project's results.

Features that need version 4.0.0 of PyMCU and the DFM firmware

pyflic analyses recordings from current FLIC systems today. The features below use the extra information recorded by **version 4.0.0 of PyMCU and the DFM firmware, which will be released in the first quarter of 2027**. Both the MCU and the DFMs must be updated to version 4.0.0 to use them.

- **Exact light timing:** how long each well's LED was on, to the millisecond, in place of samples taken five times a second.
- **Light programs recorded with the data:** each data row records which step of the light program, and which rule (*Paradigm*), was in force, so results can be split by program step.
- **Each fly's progress through the program:** its Acclimation period, the light periods it earned, and, in ratio assays, its current requirement and progress toward it. The end of Progressive Ratio training is read directly from these records.
- **Rule checks:** a check that each Paradigm behaved as programmed, so that you know the flies experienced the light schedule you intended.

The FLIC R Code will not support the new features of version 4.0.0. These will include:

- **Light rules named explicitly:** each rule (*Paradigm*) is written by name, with its own settings, in a plain-text Program file, where older firmware inferred the rule from combinations of settings. Six are available: Open Loop, Closed Loop, Fixed Interval, Non-Feeding, Progressive Ratio and Fixed Ratio.
- **New ratio assays:** Fixed Ratio lights the LED on every *n*-th feeding bout, and Progressive Ratio now counts separate feeding bouts and can follow an arithmetic or an exponential series.
- **Acclimation:** Fixed Interval and the ratio rules can start with a set number of closed-loop light activations, counted for each well, so every fly receives exactly the same number before the main rule begins.
- **Consistent feeding bouts:** a fly that dips in and out of the food within a set gap counts as making one bout.
- **Progress that carries over:** a well keeps its place in a rule when the next program step uses the same rule, so thresholds can change in the middle of an experiment without restarting a progressive ratio.

- **A full record of the light:** every 200 ms, the data file records the program step, cycle and Paradigm in force and, for each well, its phase (Acclimation or main), its light activations, its ratio requirement and progress, and its LED on-time to the millisecond.

To analyse experiments that use these features, use pyflic.

Further information

For more detailed information about the FLIC system and these analysis options, please contact **Flidea** at info@flidea.tech.

For questions about the analysis code itself, please contact its primary author, **Scott Pletcher**, at spletch@umich.edu.